

Primero aclaremos qué NO es un agente

- ✗ NO es ChatGPT respondiéndote más cosas
- ✗ NO es un prompt más largo o más complejo
- ✗ NO es un chatbot que "razona"
- ✗ NO es marketing tech para vender más IA

Todo eso es lo que vende el marketing. Vamos a lo serio.



Un agente es algo concreto Una pieza de software que

- 1** Tiene un **OBJETIVO** específico
- 2** Conoce el **ESTADO** del sistema
- 3** Toma **DECISIONES** dentro de límites definidos
- 4** Ejecuta **ACCIONES** con herramientas concretas



Cómo funciona un agente dentro de un sistema

El ciclo básico que ejecuta



Ejemplo real:

Pedido "pagado" → ¿se puede enviar? → **enviar pedido** → **Pedido "enviado"**

Cada paso es trazable, auditable, y reversible si hace falta.

No es lo mismo USAR IA que INTEGRARLA en un sistema

USAR IA (prompts)

- × Prompt → respuesta
- × Iteración sin estructura
- × Decisiones manuales
- × Sin contexto del sistema
- × El humano integra todo

INTEGRAR IA (agentes)

- ✓ Estado → decisión → acción
- ✓ Contexto persistente
- ✓ Decisiones automatizadas
- ✓ Bajo reglas e invariantes
- ✓ El sistema se autogestiona

Cómo se construye un agente en la práctica

- 1 Defines primero el sistema**
Estados, transiciones, reglas (esto es SDD del Día 1)
- 2 Identificas qué decisiones hay que tomar**
Dentro del flujo del sistema, dónde hay decisiones
- 3 Asignas agentes a esas decisiones**
Un agente por punto de decisión, no agentes mágicos
- 4 Limitas qué puede hacer cada agente**
Herramientas concretas + guardrails (MCP entra aquí)



Y aquí entra MCP

Model Context Protocol

M C P

Un estándar abierto que define EXACTAMENTE
a qué herramientas y datos puede acceder un agente
Es la pieza que permite que la IA sea segura en producción

¿Qué soluciona MCP en la práctica?

SIN MCP

- × Le das credenciales sueltas a la IA
- × La IA "puede" hacer lo que quiera con ellas
- × No hay registro estandarizado de qué hace
- × Si cambia el modelo, todo se rompe
- × Cada integración es código custom

CON MCP

- ✓ Defines herramientas concretas que la IA puede usar
- ✓ Cada herramienta tiene contrato de entrada/salida
- ✓ Cada llamada queda registrada y es auditable
- ✓ El modelo es intercambiable (Claude, GPT, etc.)
- ✓ Estándar abierto = ecosistema compartido

Tipos de agentes que vas a construir

Los más comunes en sistemas reales

- 1 Validador** *Valida reglas de negocio antes de aprobar una operación*
- 2 Clasificador** *Clasifica entradas (incidencias, leads, documentos)*
- 3 Generador** *Genera código en base a especificaciones*
- 4 Revisor** *Revisa y valida resultados de otros agentes*
- 5 Coordinador** *Coordina distintos pasos del flujo entre agentes*

Cómo se GOBIERNAN los agentes

Sin gobernanza, los agentes son una bomba de relojería en producción

G

Guardrails

Límites duros: qué NO puede hacer, sin excepciones

C

Contratos

Entrada y salida definidas (estructura, tipos, formato)

T

Trazabilidad

Cada decisión registrada: qué entró, qué salió, por qué

E

Confidence threshold

Si confianza < umbral → escala a humano



Y aquí se cierra el círculo

SDD + Agentes

SDD define el sistema. Los agentes lo ejecutan.

Sin SDD, los agentes son caos.
Sin agentes, SDD es solo documentación.

Juntos, son el nuevo modelo de ingeniería profesional.



Y todo esto, en la práctica...

Vamos a verlo EN DIRECTO

Andrés León

Lead Técnico SDD · +25 años de experiencia

Va a construir un sistema completo con SDD + Agentes desde cero

¿Cómo se pasa de uno a otro?

Spec-Driven Development

+ Agentes + Gobernanza de IA

Esto es lo que diferencia los dos perfiles

Y esto es lo que enseñamos en el Máster

Bloque 1

Fundamentos de Ingeniería de Software Profesional en la Era de la IA

Este bloque establece la línea base profesional común sobre la que se construye todo el máster.

No es un repaso de conceptos básicos ni aprender lenguajes.

Es el ajuste necesario para empezar a trabajar con estándar profesional. Con la IA acelerando todo, el problema ya no es saber más herramientas. El problema es trabajar sin método.

Aquí se alinean tres elementos clave: **Mentalidad de Ingeniería** · **Base Técnica Sólida** · **Criterio para usar IA**

MÓDULO 1.1 - Fundamentos de entorno y control técnico

El entorno deja de ser algo secundario y pasa a ser parte del sistema. Se trabaja:

- Uso profesional de terminal y tooling
- Git como sistema de trazabilidad real (no solo commits sueltos)
- Flujos de trabajo colaborativos
- Estructura de repositorios mantenibles

Control real sobre el código y el proceso de trabajo.

MÓDULO 1.2 - Lenguajes como herramientas (no como identidad)

Se abandona el enfoque de "especializarse en un lenguaje". Se entrena el criterio para usar cada herramienta según contexto:

- Python → rapidez y experimentación
- Java → sistemas estructurados y robustez empresarial
- TypeScript → backend moderno tipado
- C & Rust → control de rendimiento y memoria

Entender fortalezas, límites y uso real de cada stack.

</codecrypto.academy>

MÓDULO 2.3 - Construcción y verificación

- Planificación por slices verticales para reducir riesgo
- Clean Code y SOLID en la práctica
- Testing como verificación de contratos (no "a ver si pasa")
- Contract testing para integraciones
- CI/CD como auditor automático

Resultado: flujo profesional donde el sistema se protege solo.

MÓDULO 2.4 - Observabilidad y operación

- Métricas derivadas de lo que la spec prometió
- SLOs / SLIs y error budgets con criterio
- Telemetría estructurada: logs, métricas y trazas
- Runbooks operativos activados por alertas
- Despliegue controlado: feature flags, canary releases

Resultado: sistemas que se pueden operar de verdad, no solo enseñar.

MÓDULO 2.5 - Plataforma e infraestructura

- Docker y contenedores como base reproducible
- Cloud con criterio técnico según restricciones
- Terraform: infraestructura versionada y repetible
- CI/CD avanzada y gestión segura de secretos
- Entornos de despliegue y promoción de artefactos

Resultado: despliegues replicables, sin "en mi máquina funciona".

</codecrypto.academy>

Proyecto 3 | SEGUROS Foco: reglas duras + IA integrada con contratos

Web admin + web cliente: alta de pólizas, gestión de coberturas, declaración de siniestros mediante conversación asistida por IA, gestión simple de expedientes.

La IA se integra para guiar la declaración del siniestro, con contratos claros de entrada/salida y validación de outputs.

Tecnologías: IA gobernada con MCP, testing de contratos de IA, observabilidad.

Proyecto 4 | MARKETPLACE / SaaS Foco: conectar ingeniería, operación y negocio

Web admin + web cliente: planes, suscripciones, facturación, upgrades, downgrades, cancelaciones y métricas de negocio.

El proyecto conecta la ingeniería con la operación y el negocio real: SLOs, métricas de retención y coste por operación.

Tecnologías: arquitectura de suscripciones, métricas derivadas de la spec, CI/CD avanzado.

Al finalizar este bloque, el alumno:

- Alentaa un dominio nuevo sin improvisar
- Deriva arquitectura desde restricciones reales
- Construye sistemas verificables y operables
- Aplica el mismo método en distintos dominios
- Integra IA bajo control
- Defiende decisiones con criterio profesional

No aprendes recetas. Aprendes método.

</codecrypto.academy>

Antes y Después

ANTES — Trabajas escribiendo código

- Empiezas por la implementación sin una base clara
- Tomas decisiones por experiencia o intuición
- La arquitectura evoluciona sobre la marcha
- Utilizas IA, pero sin control sobre su integración
- Validar el sistema depende de pruebas puntuales
- La operación se plantea cuando el sistema ya está construido

Resultado: sistemas que funcionan, pero son difíciles de explicar, verificar o mantener.

DESPUÉS — Trabajas diseñando y construyendo sistemas

- Defines qué hay que construir antes de escribir código
- Tomas decisiones a partir de restricciones reales
- La arquitectura responde a una lógica clara y documentada
- Integras la IA como componente bajo control
- Cada parte del sistema se puede verificar
- La operación se diseña desde el inicio

Resultado: sistemas que no solo funcionan. Se entienden, se pueden verificar y se pueden operar.

A partir de aquí, la diferencia no está en lo que sabes. Está en cómo trabajas.